

Chapter 13. Moving On from Here

Congratulations on getting this far. You should now have a basis for understanding the issues involved in many areas of programming. Even if you never use assembly language again, you have gained a valuable perspective and mental framework for understanding the rest of computer science.

There are essentially three methods to learn to program:

- From the Bottom Up - This is how this book teaches. It starts with low-level programming, and works toward more generalized teaching.
- From the Top Down - This is the opposite direction. This focuses on what you want to do with the computer, and teaches you how to break it down more and more until you get to the low levels.
- From the Middle - This is characterized by books which teach a specific programming language or API. These are not as concerned with concepts as they are with specifics.

Different people like different approaches, but a good programmer takes all of them into account. The bottom-up approaches help you understand the machine aspects, the top-down approaches help you understand the problem-area aspects, and the middle approaches help you with practical questions and answers. To leave any of these aspects out would be a mistake.

Computer Programming is a vast subject. As a programmer, you will need to be prepared to be constantly learning and pushing your limits. These books will help you do that. They not only teach their subjects, but also teach various ways and methods of *thinking*. As Alan Perlis said, "A language that doesn't affect the way you think about programming is not worth knowing"

(<http://www.cs.yale.edu/homes/perlis-alan/quotes.html>). If you are constantly looking for new and better ways of doing and thinking, you will make a successful programmer. If you do not seek to enhance yourself, "A little sleep, a little slumber, a little folding of the hands to rest - and poverty will come on you like a

Chapter 13. Moving On from Here

bandit and scarcity like an armed man." (Proverbs 24:33-34 NIV). Perhaps not quite that severe, but still, it's best to always be learning.

These books were selected because of their content and the amount of respect they have in the computer science world. Each of them brings something unique. There are many books here. The best way to start would be to look through online reviews of several of the books, and find a starting point that interests you.

From the Bottom Up

This list is in the best reading order I could find. It's not necessarily easiest to hardest, but based on subject matter.

- *Programming from the Ground Up* by Jonathan Bartlett
- *Introduction to Algorithms* by Thomas H. Cormen, Charles E. Leiserson, and Ronald L. Rivest
- *The Art of Computer Programming* by Donald Knuth (3 volume set - volume 1 is the most important)
- *Programming Languages* by Samuel N. Kamin
- *Modern Operating Systems* by Andrew Tanenbaum
- *Linkers and Loaders* by John Levine
- *Computer Organization and Design: The Hardware/Software Interface* by David Patterson and John Hennessy

From the Top Down

These books are arranged from the simplest to the hardest. However, they can be read in any order you feel comfortable with.

- *How to Design Programs* by Matthias Felleisen, Robert Bruce Findler, Matthew Flatt, and Shriram Krishnamurthi, available online at <http://www.htdp.org/>
- *Simply Scheme: An Introduction to Computer Science* by Brian Harvey and Matthew Wright
- *How to Think Like a Computer Scientist: Learning with Python* by Allen Downey, Jeff Elkner, and Chris Meyers, available online at <http://www.greenteapress.com/thinkpython/>
- *Structure and Interpretation of Computer Programs* by Harold Abelson and Gerald Jay Sussman with Julie Sussman, available online at <http://mitpress.mit.edu/sicp/>
- *Design Patterns* by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides
- *What not How: The Rules Approach to Application Development* by Chris Date
- *The Algorithm Design Manual* by Steve Skiena
- *Programming Language Pragmatics* by Michael Scott
- *Essentials of Programming Languages* by Daniel P. Friedman, Mitchell Wand, and Christopher T. Haynes

From the Middle Out

Each of these is the best book on its subject. If you need to know these languages, these will tell you all you need to know.

- *Programming Perl* by Larry Wall, Tom Christiansen, and Jon Orwant
- *Common LISP: The Language* by Guy R. Steele
- *ANSI Common LISP* by Paul Graham
- *The C Programming Language* by Brian W. Kernighan and Dennis M. Ritchie

Chapter 13. Moving On from Here

- *The Waite Group's C Primer Plus* by Stephen Prata
- *The C++ Programming Language* by Bjarne Stroustrup
- *Thinking in Java* by Bruce Eckel, available online at <http://www.mindview.net/Books/TIJ/>
- *The Scheme Programming Language* by Kent Dybvig
- *Linux Assembly Language Programming* by Bob Neveln

Specialized Topics

These books are the best books that cover their topic. They are thorough and authoritative. To get a broad base of knowledge, you should read several outside of the areas you normally program in.

- Practical Programming - *Programming Pearls* and *More Programming Pearls* by Jon Louis Bentley
- Databases - *Understanding Relational Databases* by Fabian Pascal
- Project Management - *The Mythical Man-Month* by Fred P. Brooks
- UNIX Programming - *The Art of UNIX Programming* by Eric S. Raymond, available online at <http://www.catb.org/~esr/writings/taoup/>
- UNIX Programming - *Advanced Programming in the UNIX Environment* by W. Richard Stevens
- Network Programming - *UNIX Network Programming* (2 volumes) by W. Richard Stevens
- Generic Programming - *Modern C++ Design* by Andrei Alexandrescu
- Compilers - *The Art of Compiler Design: Theory and Practice* by Thomas Pittman and James Peters

- Compilers - *Advanced Compiler Design and Implementation* by Steven Muchnick
- Development Process - *Refactoring: Improving the Design of Existing Code* by Martin Fowler, Kent Beck, John Brant, William Opdyke, and Don Roberts
- Typesetting - *Computers and Typesetting* (5 volumes) by Donald Knuth
- Cryptography - *Applied Cryptography* by Bruce Schneier
- Linux - *Professional Linux Programming* by Neil Matthew, Richard Stones, and 14 other people
- Linux Kernel - *Linux Device Drivers* by Alessandro Rubini and Jonathan Corbet
- Open Source Programming - *The Cathedral and the Bazaar: Musings on Linux and Open Source by an Accidental Revolutionary* by Eric S. Raymond
- Computer Architecture - *Computer Architecture: A Quantitative Approach* by David Patterson and John Hennessy

Further Resources on Assembly Language

In assembly language, your best resources are on the web.

- <http://www.linuxassembly.org/> - a great resource for Linux assembly language programmers
- <http://www.sandpile.org/> - a repository of reference material on x86, x86-64, and compatible processors
- <http://www.x86.org/> - Dr. Dobb's Journal Microprocessor Resources
- <http://www.drpaulcarter.com/pcasm/> - Dr. Paul Carter's PC Assembly Language Page
- <http://webster.cs.ucr.edu/> - The Art of Assembly Home Page

Chapter 13. Moving On from Here

- <http://www.intel.com/design/pentium/manuals/> - Intel's manuals for their processors
- <http://www.janw.easynet.be/> - Jan Wagemaker's Linux assembly language examples
- <http://www.azillionmonkeys.com/qed/asm.html> - Paul Hsieh's x86 Assembly Page